

Java Programming • Master 1

TP1: Install JDK and write HelloWorld program.
TP2: Create programs using variables and arithmetic operators.
TP3: Write programs using if/else and loops.
TP4: Create reusable methods and simple utilities.
TP5: Design a Student class and create objects.
TP6: Implement encapsulation using getters and setters.
TP7: Create inheritance hierarchy Person → Student → Professor.
TP8: Demonstrate polymorphism using Animal classes.
TP9: Implement Shape interface with Circle and Rectangle.
TP10: Write programs using try/catch exception handling.
TP11: Manage collections using ArrayList and HashMap.
TP12: Develop a mini project (library or student management system).

Main goals

- Master Java syntax and object-oriented programming.
- Develop modular and maintainable applications.
- Use collections, exceptions and interfaces.
- Build a small Java application as a final project.

Module 1 · Java Foundations

Environment, execution model, first program, and the conventions that make Java portable and readable.

Selected themes

- Course Objectives and Learning Outcomes
- What Is Java?
- A Brief History of Java
- Java Philosophy: Write Once, Run Anywhere
- Java Virtual Machine (JVM)

Course Objectives and Learning Outcomes

Concept

Understand Java basics, environment, and program structure.

Java is an object-oriented programming language designed for portability.

Created by James Gosling at Sun Microsystems in 1995.

Write Once, Run Anywhere using the JVM.

Java Virtual Machine executes compiled bytecode.

Focus

- Understand Java basics, environment, and program structure.
- This module builds the mental model needed to compile, run, and reason about Java programs.

JDK vs. JRE

Concept

JDK (Java Development Kit)

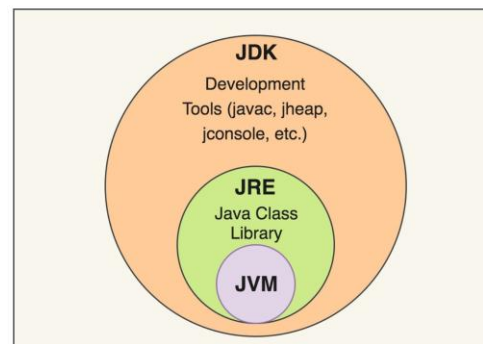
- Used to develop, compile, debug Java programs
- Includes: javac, debugger, tools + JRE

JRE (Java Runtime Environment)

- Used to run Java applications
- Includes: JVM + standard libraries

JVM (Java Virtual Machine)

- Executes bytecode



JDK includes tools **for** development; JRE only runs programs.

Focus

Java source code (.java) →
bytecode (.class) →
JVM execution

Hello World: Complete Program

- Observe the class declaration and the required main method signature.
- Execution starts in main(), which the JVM searches for when launching the program.
- System.out.println(...) sends text to the standard output stream.

A Java program is organized into classes.

Formatted Java code

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World");  
4     }  
5 }
```

Sequential Program Flow

Concept

- Execution begins in `main()` and follows statements sequentially.
- `//` single line comment
- `/*` multi-line `*/` comments
- Use indentation and naming conventions
- Use Javadoc to document classes and methods.

Execution begins in `main()` and follows statements sequentially.

Common Beginner Errors in Java

Missing semicolon, incorrect class name.

Laboratory Preparation: Installing the JDK

Concept

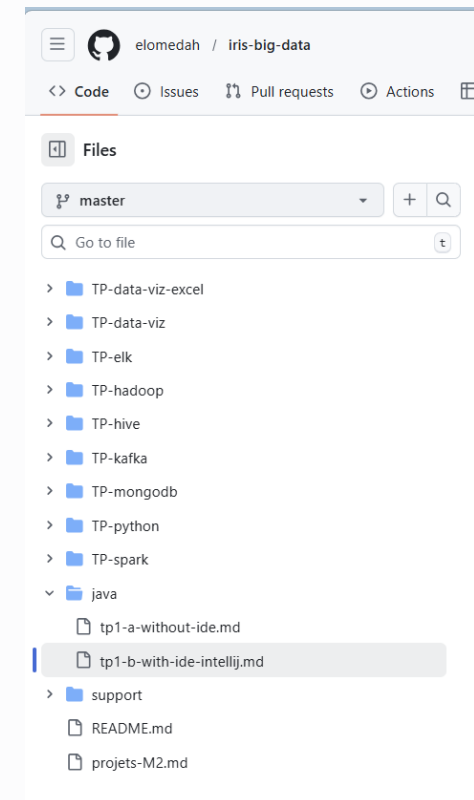
Install JDK and test first program.

[iris-big-data/java at master · elomedah/iris-big-data](#)

[iris-big-data/java/tp1-a-without-ide.md at master · elomedah/iris-big-data](#)

[iris-big-data/java/tp1-b-with-ide-intellij.md at master · elomedah/iris-big-data](#)

Focus



Module 2 · Variables, Types, and Operators

- Variables
- Primitive types
- Operators
- Casting
- Constants

Focus

- Variables: Storing Data During Execution
- Variable Declaration Syntax
- Variable Declaration: Worked Example
- Primitive Data Types in Java
- Integer Variable Example

Variables: Storing Data During Execution

Concept

Variables store values used during execution.

```
int age = 20;
```

```
Integer age = 20;
```

Syntax

```
Type variableName;
```

Primitive Data Types in Java

Concept

byte, short, int, long, float, double, char, boolean.

Difference between float and double

Type	Size	Precision
float	32 bits	~6–7 decimal digits
double	64 bits	~15–16 decimal digits

Syntax

```
// Integer types
byte smallNumber = 100;
short mediumNumber = 10000;
int number = 100000;
long largeNumber = 10000000L;

// Floating-point types
float price = 19.99f;
double pi = 3.1415926535;

// Character type
char letter = 'A';

// Boolean type
boolean isValid = true;
```

Type Casting

Concept

Converting values between types.

Java code

```
double x = 10.5;  
int y = (int)x;
```

Operators in Java: Overview

Concept

Operators perform operations on variables.

Arithmetic Operators

+ - * / %

Relational Operators

== != > < >= <=

Logical Operators

&& || !

Increment and Decrement Operators

++ --

Syntax cue

```
int a = 10;  
int b = 5;  
int c = a + b;
```

Expressions

Concept

```
int result = (a + b) * 2;
```

Syntax

```
1 int result = (a + b) * 2;
```

Focus

Constants with the final Keyword

Concept

A **constant** is a variable whose value **cannot be changed after initialization**.

final keyword defines constants.

Code

```
final type NAME = value;
```

```
final double PI = 3.14159;  
final int MAX_USERS = 50;
```

Variable Naming Conventions

Concept

camelCase for variables.

- The **first word starts with a lowercase letter**
- Each following word starts with an **uppercase letter**
- No spaces or underscores

Naming conventions for **classes (PascalCase)**

Naming for **constants (UPPER_CASE)**

Code

```
int age;  
double accountBalance;  
String firstName;  
boolean isValid;
```

String

Concept

String is an object in Java, not a primitive.

Strings are immutable: once created, cannot be changed.

String operations return a new String object.

Méthode	Description
<code>length()</code>	longueur de la chaîne
<code>toUpperCase()</code>	convertir en majuscules
<code>toLowerCase()</code>	convertir en minuscules
<code>trim()</code>	enlever les espaces
<code>charAt(i)</code>	accéder à un caractère
<code>substring()</code>	extraire une partie
<code>replace()</code>	remplacer du texte
<code>contains()</code>	vérifier présence
<code>equals()</code>	comparer deux chaînes
<code>split()</code>	découper en tableau

Code

```
String name = "Alice";  
String city = "Paris";  
  
String message = name + city;
```

Variables and Memory Allocation

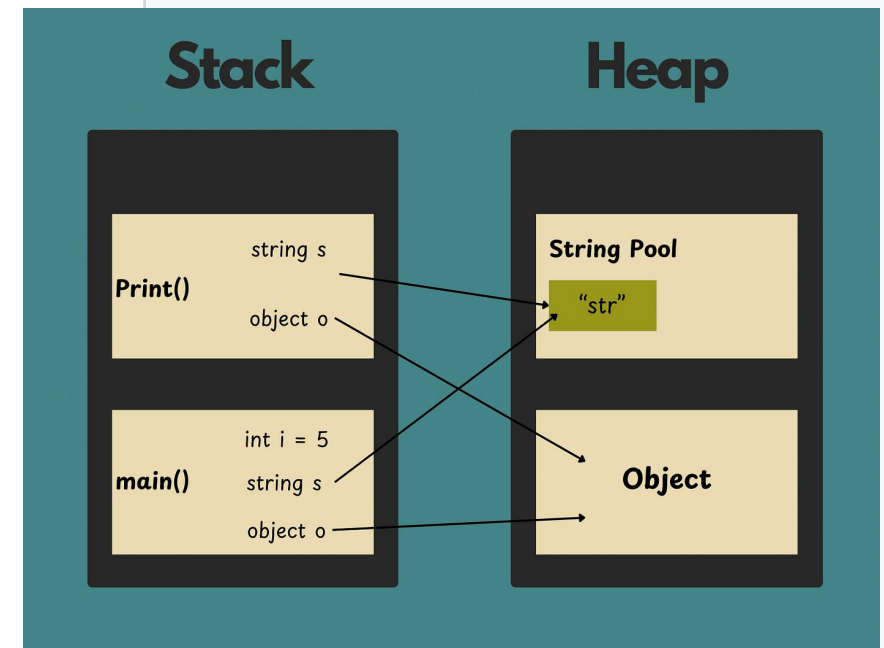
Concept

Variables occupy memory during execution.

Primitive types : direct storage

Reference types : store addresses

Stack and heap manage different kinds of data



Practice Problem

TP 2

[iris-big-data/java/tp2.md at master · elomedah/iris-big-data](#)

Module 3 · Control Flow, Methods, and Arrays

Decision structures, loops, methods, recursion, arrays, and the disciplined organization of procedural logic.

Focus

- Defining a Simple Student Class
- Conditional Logic with if / else
- Multi-Branch Decisions with else if
- Discrete Selection with switch
- Repetition with the while Loop

Defining a Simple Student Class

Concept

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Conditional Logic with if / else

Concept

Tests a boolean condition and branches execution.

if → first condition

else if → additional conditions (not mandatory)

else → default case (not mandatory)

Conditions are checked in order

Once a condition is true → the rest are skipped

The else block is optional, but useful

Syntax

```
if (condition) {  
    ...  
} else if (condition) {  
    ...  
} else {  
    ...  
}
```

```
public class DayExample {  
    public static void main(String[] args) {  
        int day = 3;  
        if (day == 1) {  
            System.out.println("Monday");  
        } else if (day == 2) {  
            System.out.println("Tuesday");  
        } else if (day == 3) {  
            System.out.println("Wednesday");  
        } else {  
            System.out.println("Other day");  
        }  
    }  
}
```

Discrete Selection with switch

Concept

The switch statement is used when you want to **select one option among many discrete values** (like numbers or strings).

- **break** stops execution of the switch
- Without **break**, Java continues to the next case (fall-through)
- **default** is optional but recommended

Syntax

```
switch (variable) {  
  case value1 :  
    // code  
    break;  
  case value2:  
    // code  
    break;  
  default :  
    // code if no case matches
```

Syntax

```
switch (day) {  
  case 1 :  
    System.out.println("Mon");  
    break;  
  case 2:  
    System.out.println("Tue");  
    break;  
  default :  
    System.out.println("Other");
```

Repetition with the while Loop

Concept

Repeats a block while a condition is true.

- The condition is evaluated before each iteration
- If the condition is true, the loop runs
- If the condition is false, the loop stops

```
while (condition) {  
    // instructions to repeat  
}
```

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

for Loop: Counter-Controlled Iteration

Concept

Repeats a block while a condition is true.

- The initializer executes once before the loop begins.
 - The condition is checked before each iteration.
 - The update expression advances the counter after each pass.
-
- Ideal when number of iterations is known
 - Compact structure (all logic in one line)
 - Improves readability for counting tasks

```
for (initialization; condition; update) {  
    // instructions to repeat  
}
```

```
for (int i=0;i<5;i++){  
    System.out.println(i);  
}
```

The do / while Loop

Concept

Executes the block at least once before checking the condition.

The do-while loop is a **post-test loop**, meaning the condition is checked **after** executing the block.

This guarantees that the code inside the loop runs **at least once**, even if the condition is false

```
do {  
    // instructions to execute  
} while (condition);
```

```
int n = 0;  
do {  
    System.out.println(n);  
    i++;  
} while (n < 3);
```

Loop Control with break and continue

Concept

- **break** exits the loop immediately.
- **continue** skips the rest of the current iteration.

Code

```
for (int i=0;i<10;i++) {  
    if (i==5) break;  
    if (i%2==0) continue;  
    System.out.println(i);  
}  
  
// prints 1,3
```

Nested Loops

Concept

A loop inside another loop
Useful for matrices.

```
for (int r=0;r<3;r++) {  
    for (int c=0;c<3;c++) {  
        System.out.print(r+","+c+" ");  
    }  
}
```

Defining and Calling Methods

A method must be defined before it is called (within class scope)

If the return type is not void, you must return a value

Parameter types must match when calling

Code

```
returnType name(params) {  
    // body  
  
    // optional (depends on returnType)  
    return value;  
}
```

```
int add(int a, int b) {  
    return a+b;  
}  
  
int result = add(1,4);
```

Void Methods

Concept

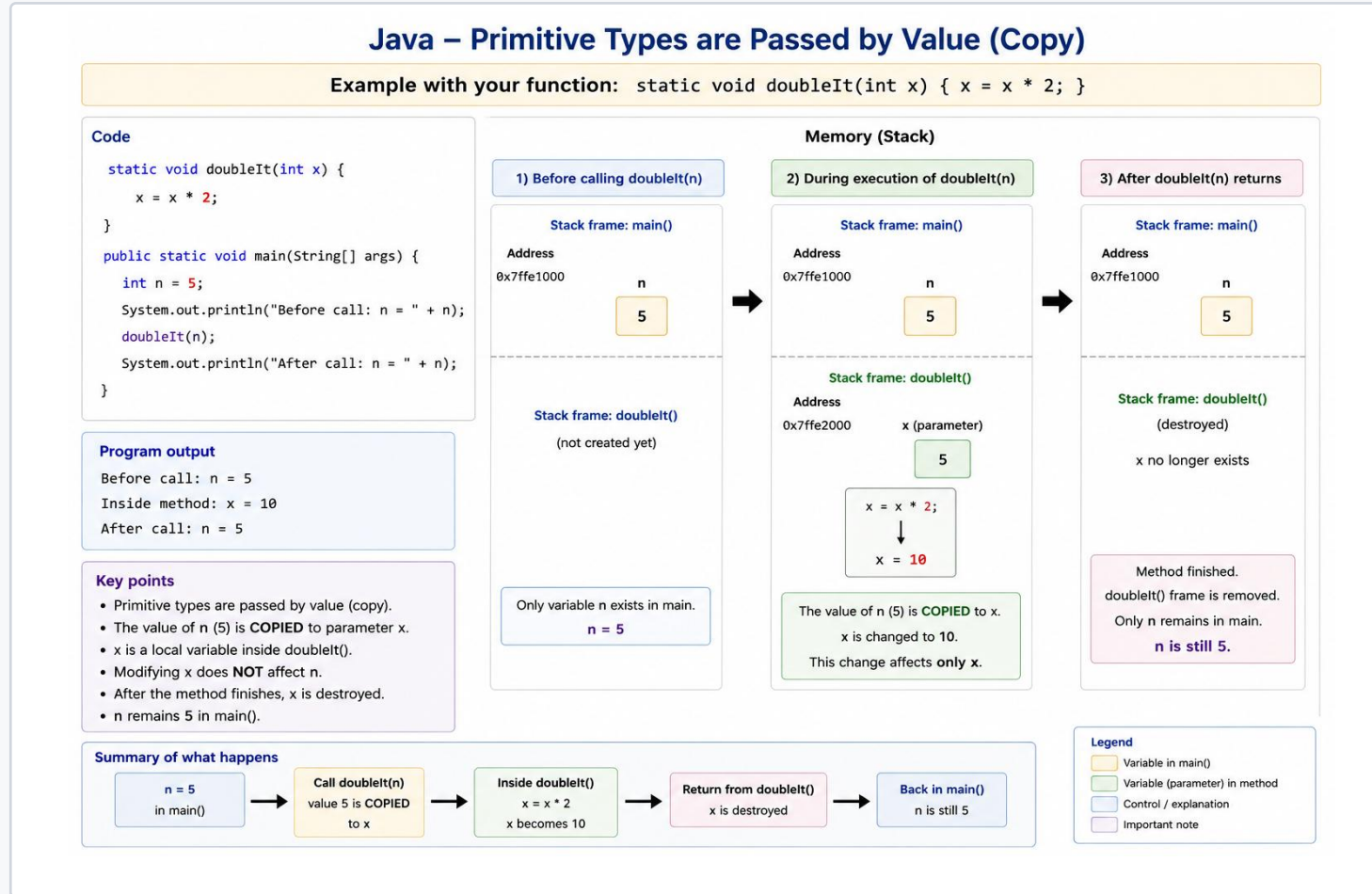
A method that performs an action but returns nothing.

- The keyword `void` means no return value
- You can use `return;` to exit early (optional)

Code

```
void greet(String name) {  
    System.out.println("Hello, " + name);  
}  
  
greet("Alice"); // prints Hello, Alice
```

Method Parameters and Pass-by-Value, Variable Scope



Code

```
static void doubleIt(int x) {
    x = x * 2;
}

int n = 5;
doubleIt(n);
// n is still 5
```

Parameters are local variables inside the method.
Primitive types are passed by value (copy).

Variables declared inside a method are local.
They cannot be accessed outside that method.
Block scope: variable inside {} only lives there.
Avoid naming conflicts between local and field vars.

Method Overloading

Same method name, different parameter types or counts.

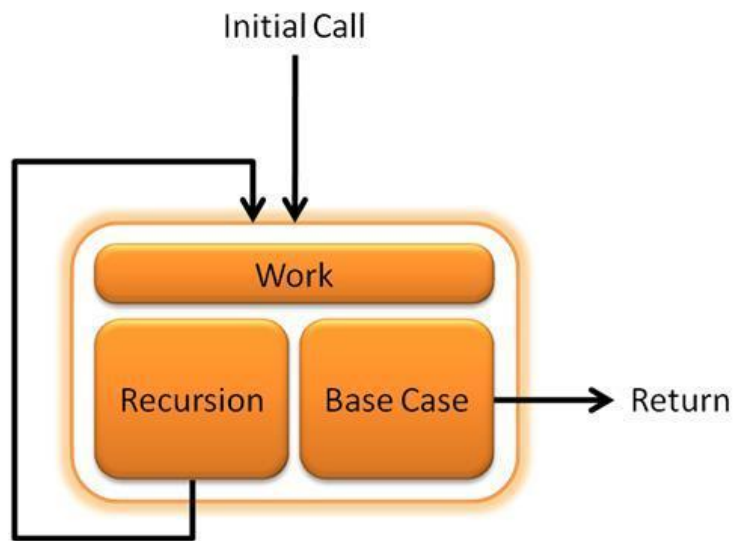
Code

```
static int add(int a, int b) {  
    return a+b;  
}  
  
static double add(double a, double b) {  
    return a+b;  
}
```

Introduction to Recursion

Concept

A method that calls itself to solve smaller sub-problems.



Code

```
static int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

Arrays: Fixed-Size Collections

Concept

Fixed-size container of elements of the same type.

```
int[] scores = new int[5];
```

```
scores[0] = 90;
```

Access via index (0-based): scores[0] to scores[4].

Length: scores.length → 5

```
1 int[] scores = new int[5];  
2 scores[0] = 90;
```

Array Initialization in One Statement

Concept

Declare and populate in one line.

```
String[] days = {"Mon","Tue","Wed","Thu","Fri"};
```

```
System.out.println(days[2]); // Wed
```

Iterate with for or for-each loop.

Code cue

```
1 String[] days =  
  {"Mon", "Tue", "Wed", "Thu", "Fri"};  
2 System.out.println(days[2]); //  
  Wed
```

Enhanced for Loop

Concept

Simplified loop for arrays and collections.

```
int[] nums = {1, 2, 3, 4, 5};
```

```
for (int n : nums) {
```

```
    System.out.println(n);
```

```
}
```

Cannot modify the array element directly.

Code cue

```
int[] nums = {1, 2, 3, 4, 5};  
for (int n : nums) {  
    System.out.println(n);  
}
```

Module 4 · Strings, Utilities, and Input/Output

Strings, utility classes, formatted I/O, arrays as parameters, wrappers, and robust console interaction.

- ArrayList: Creating and Populating a List
- Static Methods Revisited
- Using the Math Utility Class
- Understanding the String Class
- Common String Methods

Static Methods Revisited

Concept

Static methods belong to the class, not an instance.

Called as `ClassName.method()` or just `method()` within same class.

Code cue

```
static double circleArea(double r) {  
    return Math.PI * r * r;  
}
```

Using the Math Utility Class

Concept

`java.lang.Math` provides common math operations.

`Math.abs(-5) → 5`

`Math.max(3, 7) → 7`

`Math.sqrt(16) → 4.0`

`Math.pow(2, 8) → 256.0`

Code cue

```
1 Math.abs(-5) → 5
2 Math.max(3, 7) → 7
3 Math.sqrt(16) → 4.0
4 Math.pow(2, 8) → 256.0
```

Teaching focus

- `java.lang.Math` provides common math operations.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Understanding the String Class

Concept

String is an object in Java, not a primitive.

String name = "Alice";

Strings are immutable: once created, cannot be changed.

String operations return a new String object.

Code

```
String name = "Alice";  
// return a String object.
```

- String is an object in Java, not a primitive.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Common String Methods

Concept

`name.length()` → number of characters.

`name.toUpperCase()` → "ALICE"

`name.substring(1, 3)` → "li"

`name.contains("lic")` → true

`name.replace('l','L')` → "ALice"

- `name.length()` → number of characters.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Concatenating Strings

Concept

Use + to join strings.

```
String full = "Hello" + " " + "World";
```

Mixed types: "Age: " + 20 → "Age: 20"

Use StringBuilder for many concatenations (efficient).

Code

```
String full = "Hello" + " " + "World";
```

- Use + to join strings.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Comparing Strings Correctly

Concept

Never compare Strings with `==`; use `.equals()`.

`String a = "hello"; String b = "hello";`

`a == b` → may be false (reference comparison)

`a.equals(b)` → true (content comparison)

`a.equalsIgnoreCase(b)` → ignores case

Code cue

```
// Never compare Strings with ==;  
// use always .equals()  
String a = "hello";  
String b = "hello";
```

- Never compare Strings with `==`; use `.equals()`.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Building Strings with StringBuilder

Concept

Mutable sequence of characters - efficient for building strings.

```
StringBuilder sb = new StringBuilder();
```

```
sb.append("Hello");
```

```
sb.append(" World");
```

```
String result = sb.toString(); // "Hello World"
```

Code cue

```
new StringBuilder();  
sb.append("Hello");  
sb.append(" World");  
String result = sb.toString(); // "Hello World"
```

- Mutable sequence of characters - efficient for building strings.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Parsing Numbers from Strings

Concept

Convert String to numbers and back.

```
int n = Integer.parseInt("42");
```

```
double d = Double.parseDouble("3.14");
```

```
String s = String.valueOf(100);
```

Throws `NumberFormatException` if invalid input.

Code

```
int n = Integer.parseInt("42");  
double d = Double.parseDouble("3.14");  
String s = String.valueOf(100);
```

- Convert String to numbers and back.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Basic try / catch Example

Concept

try{ }

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Code

```
try{  
    int x = 10/0;  
}catch (Exception e){  
    System.out.println("Error");  
}
```

Method Design Principles

Concept

One method = one responsibility.

Keep methods short (< 20 lines as a guideline).

Use descriptive names: `calculateTotal()` not `calc()`.

Parameters should be minimal and meaningful.

Teaching focus

- One method = one responsibility.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Passing Arrays to Methods

Concept

**Arrays are passed by reference
– changes affect original**

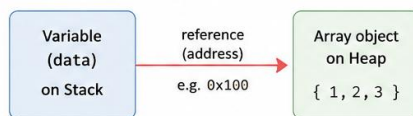
```
static void fillZeros(int[] arr) {
    for (int i = 0; i < arr.length; i++)
        arr[i] = 0;
}

public static void main(String[] args) {
    int[] data = {1, 2, 3};
    fillZeros(data);
    // all 0
}
```

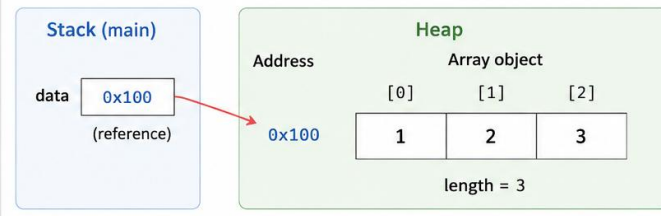
Key points

- An array variable stores a reference (address) to the array object in memory.
- When an array is passed to a method, the reference (address) is copied.
- Both the caller and the method point to the same array in the heap.
- Changing elements through the parameter affects the original array.

Overview

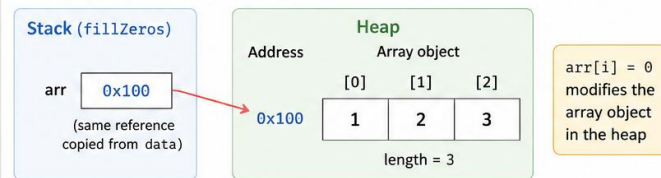


Before the call fillZeros(data)

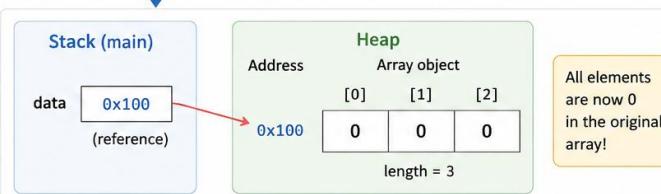


Call: fillZeros(data)

Inside fillZeros(int[] arr)



After the call (return to main)



Code cue

```
static void fillZeros(int[] arr) {
    for (int i=0; i<arr.length; i++)
        arr[i]=0;
}

int[] data={1,2,3};
fillZeros(data); // all 0
```

- Arrays are passed by reference - changes affect original.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Returning Arrays from Methods

Concept

Methods can return arrays just like any type.

```
static int[] range(int n) {  
    int[] r = new int[n];  
    for(int i=0;i<n;i++) r[i]=i;  
    return r;  
}
```

Code

```
1 return arrays just like any  
   type.  
2 static int[] range(int n) {  
3     int[] r = new int[n];  
4     for (int i=0;i<n;i++) r[i]=i;  
5     return r;
```

Teaching focus

- Methods can return arrays just like any type.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Variable-Length Arguments (varargs)

Concept

Accept variable number of arguments with ...

```
static int sum(int... nums) {
```

```
    int total = 0;
```

```
    for (int n : nums)
```

```
        total += n;
```

```
    return total;
```

```
}
```

sum(1,2,3,4) → 10

Code cue

```
1 static int sum(int... nums) {  
2     int total = 0;  
3     for (int n : nums) total += n;  
4     return total;
```

- Accept variable number of arguments with ...
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Wrapper Classes for Primitive Types

Concept

Each primitive has an object wrapper.

int → Integer,

double → Double,

boolean → Boolean.

Integer.MAX_VALUE → 2147483647

Used in collections (ArrayList<Integer>).

Autoboxing:

```
int n = 5;
```

```
Integer obj = n; // automatic
```

Code

```
ArrayList<Integer>  
int n = 5; Integer obj = n; //  
automatic
```

Teaching focus

- Each primitive has an object wrapper.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Autoboxing and Unboxing

Concept

Java automatically converts primitives ↔ wrappers.

Integer x = 10; // autoboxing (int → Integer)

int y = x; // unboxing (Integer → int)

Useful in collections but has performance overhead.

Code

```
1 Integer x = 10; // autoboxing
  (int → Integer)
2 int y = x;      // unboxing
  (Integer → int)
```

- Java automatically converts primitives ↔ wrappers.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Generating Random Numbers

Concept

Use `java.util.Random` for random values.

Code

```
1 Use java.util.Random for random values.  
2 Random rand = new Random();  
3 int n = rand.nextInt(100); // 0 to 99  
4 double d = rand.nextDouble(); // 0.0 to 1.0  
5 Or: (int)(Math.random() * 100)
```

Reading Console Input with Scanner

Concept

java.util.Scanner reads input from the console.

- Read the example from top to bottom and identify the role of each line.
- Relate the syntax to the concept introduced by the slide title.
- Be ready to adapt this pattern during a practical laboratory session.

Formatted Java code

```
1 java.util.Scanner reads input from the console.  
2 Scanner sc = new Scanner(System.in);  
3 System.out.print("Enter name: ");  
4 String name = sc.nextLine();  
5 int age = sc.nextInt();
```

Formatted Output with printf

Concept

System.out.printf formats output like C printf.

```
printf("Name: %s-10s Age: %3d%n", name, age);
```

%d = integer, %f = float, %s = string, %n = newline.

%.2f formats a double to 2 decimal places.

Code cue

```
1 System.out.printf formats  
output like C printf.  
2 printf("Name: %s-10s Age:  
%3d%n", name, age);
```

- System.out.printf formats output like C printf.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Creating Formatted Strings

Concept

Creates a formatted String without printing it.

```
String msg = String.format("PI = %.4f", Math.PI);
```

```
// msg = "PI = 3.1416"
```

Useful for building messages before displaying them.

Code cue

```
1 String msg = String.format("PI  
= %.4f", Math.PI);
```

- Creates a formatted String without printing it.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Common Runtime Pitfalls

Concept

NullPointerException: calling a method on null reference.

```
String s = null; s.length(); // NPE!
```

Always check for null before using an object.

ArrayIndexOutOfBoundsException: accessing invalid index.

Code cue

```
String s = null;  
s.length(); // NPE!
```

- NullPointerException: calling a method on null reference.
- Strings, utility classes, and console I/O are essential for writing practical interactive programs.

Module 5 · Classes and Objects

From class design to object creation, constructors, references, lifecycle, and object-level behavior.

Selected themes

- Defining a Simple Student Class
- Defining a Class
- Creating Objects from a Class
- Constructors
- Default Constructors

Defining a Simple Student Class

Concept

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Defining a Class

Concept

A class is a blueprint for creating objects.

```
public class Car {  
    String brand;  
    int year;  
    double price;  
}
```

Code

```
public class Car {  
    String brand;  
    int year;  
    double price;  
}
```

Teaching focus

- A class is a blueprint for creating objects.
- Classes and objects translate problem domains into reusable software structures.

Creating Objects from a Class

Concept

Use new keyword to instantiate a class.

Code

```
1 Use new keyword to instantiate a class.  
2 Car myCar = new Car();  
3 myCar.brand = "Toyota";  
4 myCar.year = 2022;  
5 System.out.println(myCar.brand); // Toyota
```

Constructors

Concept

Special method called when creating an object.

```
public Car(String brand, int year) {  
    this.brand = brand;  
    this.year = year;  
}  
Car c = new Car("BMW", 2023);
```

Code

```
public Car(String brand, int year) {  
    this.brand = brand;  
    this.year = year;  
}  
new Car("BMW", 2023);
```

Teaching focus

- Special method called when creating an object.
- Classes and objects translate problem domains into reusable software structures.

Default Constructors

Concept

If no constructor is defined, Java provides a default one.
Default constructor takes no arguments.
Once you define any constructor, the default is removed.
Best practice: always define at least one constructor.

Teaching focus

- If no constructor is defined, Java provides a default one.
- Classes and objects translate problem domains into reusable software structures.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Instance Methods

Concept

Methods that operate on the object's fields.

```
public void displayInfo() {  
    System.out.println(brand + " (" + year + ")");  
}  
myCar.displayInfo(); // Toyota (2022)
```

Code

```
public void displayInfo() {  
    System.out.println(brand + " (" + year +  
        ")");  
}  
  
myCar.displayInfo(); // Toyota (2022)
```

Teaching focus

- Methods that operate on the object's fields.
- Classes and objects translate problem domains into reusable software structures.

Using the this Keyword

Concept

Refers to the current object instance.

Resolves naming conflict between field and parameter.

```
public void setBrand(String brand) {  
    this.brand = brand; // field = parameter  
}
```

Code

```
public void setBrand(String brand) {  
    this.brand = brand; // field = parameter  
}
```

Teaching focus

- Refers to the current object instance.
- Classes and objects translate problem domains into reusable software structures.

Constructor Chaining

Concept

One constructor calling another with this().

```
public Car() { this("Unknown", 2000); }
```

```
public Car(String b, int y) { this.brand=b; this.year=y; }
```

Reduces code duplication.

Code

```
public Car() { this("Unknown", 2000); }  
public Car(String b, int y) {  
    this.brand=b;  
    this.year=y;  
}
```

Teaching focus

- One constructor calling another with this().
- Classes and objects translate problem domains into reusable software structures.

Object References and Aliasing

Concept

Variables hold a reference (address) to the object.

```
Car a = new Car("Audi", 2020);
```

```
Car b = a; // same object!
```

```
b.brand = "BMW"; // a.brand is now BMW too
```

To copy: implement a copy constructor.

Code

```
1 new Car("Audi", 2020);  
2 Car b = a; // same object!  
3 b.brand = "BMW"; // a.brand is  
  now BMW too
```

Teaching focus

- Variables hold a reference (address) to the object.
- Classes and objects translate problem domains into reusable software structures.

Overriding toString()

Concept

Override toString() to get a readable representation.

Code

```
@Override
public String toString() {
    return brand + " " + year;
}

System.out.println(myCar); // Toyota 2022
```

Teaching focus

- Override toString() to get a readable representation.
- Classes and objects translate problem domains into reusable software structures.

Overriding equals()

Concept

Override equals() to compare objects by content.

Code

```
@Override
public boolean equals(Object o) {
    Car c = (Car) o;
    return this.brand.equals(c.brand) &&
        this.year == c.year;
}
```

Teaching focus

- Override equals() to compare objects by content.
- Classes and objects translate problem domains into reusable software structures.

Static Fields

Concept

Shared by all instances of the class.

```
static int carCount = 0;
```

```
public Car(...) { carCount++; }
```

Access as `Car.carCount` belongs to the class, not instance.

Useful for counters and constants.

Code cue

```
1 static int carCount = 0;  
2 public Car(...) { carCount++; }
```

Teaching focus

- Shared by all instances of the class.
- Classes and objects translate problem domains into reusable software structures.

Static Methods in Classes

Concept

Can be called without creating an object.
Cannot access instance fields/methods directly.

Code

```
public static int getCount() {  
    return carCount;  
}  
int n = Car.getCount();
```

Teaching focus

- Can be called without creating an object.
- Classes and objects translate problem domains into reusable software structures.

Working Safely with null References

Concept

A reference that points to no object.

Car c = null;

c.brand → NullPointerException!

Check: if (c != null) { c.brand = "X"; }

Use Optional<> in modern Java for cleaner null handling.

Code

```
Car c = null;  
if (c != null) { c.brand = "X"; }
```

Teaching focus

- A reference that points to no object.
- Classes and objects translate problem domains into reusable software structures.

Object Lifecycle and Garbage Collection

Concept

Objects are created with `new` and live on the heap.
Garbage Collector (GC) automatically frees unused objects.
No manual memory management needed (unlike C++).
Set reference to null to hint GC: `car = null;`

Code

```
new and live on the heap.  
Set reference to null to hint GC:  
car = null;
```

Teaching focus

- Objects are created with `new` and live on the heap.
- Classes and objects translate problem domains into reusable software structures.

Introduction to UML Class Diagrams

Concept

UML class diagram: ClassName | fields | methods.

Car

- brand: String

- year: int

+ displayInfo(): void

Helps design before coding.

Code

```
class diagram: ClassName | fields  
| methods.
```

Teaching focus

- UML class diagram: ClassName | fields | methods.
- Classes and objects translate problem domains into reusable software structures.

Overloaded Constructors

Concept

Overloaded constructors provide flexible instantiation.

Car() — default

Car(String brand) — brand only

Car(String brand, int year, double price) — full

Each serves different use cases.

Teaching focus

- Overloaded constructors provide flexible instantiation.
- Classes and objects translate problem domains into reusable software structures.

Arrays of Objects

Concept

Arrays can hold objects.

```
Car[] garage = new Car[3];  
garage[0] = new Car("Tesla", 2023);  
garage[1] = new Car("Audi", 2021);  
for (Car c : garage) System.out.println(c);
```

Code cue

```
1 new Car[3];  
2 new Car("Tesla", 2023);  
3 new Car("Audi", 2021);  
4 for (Car c : garage)  
  System.out.println(c);
```

Teaching focus

- Arrays can hold objects.
- Classes and objects translate problem domains into reusable software structures.

Comparing Objects: == versus equals()

Concept

== compares references, not content.

```
Car a = new Car("BMW", 2020);
```

```
Car b = new Car("BMW", 2020);
```

a == b → false (different objects in memory)

a.equals(b) → true (if equals() is overridden)

Code cue

```
1 new Car("BMW", 2020);  
2 new Car("BMW", 2020);
```

Teaching focus

- == compares references, not content.
- Classes and objects translate problem domains into reusable software structures.

Design Exercise: BankAccount Class

Concept

Design a BankAccount class:

Fields: owner (String), balance (double)

Constructor: BankAccount(String owner, double balance)

Methods: deposit(double), withdraw(double), getBalance()

Check for invalid amounts in withdraw.

Teaching focus

- Design a BankAccount class:
- Classes and objects translate problem domains into reusable software structures.
- Before coding, identify the class responsibilities, fields, and method contracts.

Classes and Objects: Summary

Concept

Classes: fields, constructors, methods.

Objects: instantiation, references, lifecycle.

this keyword: field vs parameter disambiguation.

Override toString() and equals().

Next: Encapsulation and access control.

Teaching focus

- Classes: fields, constructors, methods.
- Classes and objects translate problem domains into reusable software structures.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 6 · Encapsulation and Access Control

Encapsulation, access modifiers, invariants, validation, immutability, and API design for safe objects.

Selected themes

- Access Modifiers
- Why Encapsulation Matters
- Private Fields
- Getter Methods

Access Modifiers

Concept

Control who can access a class member.

public: accessible everywhere.

private: accessible only within the class.

protected: accessible within package and subclasses.

(default): accessible within the same package.

Code

```
1 class member.
```

Teaching focus

- Control who can access a class member.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Why Encapsulation Matters

Concept

Protects internal state from invalid modifications.

Without encapsulation: `account.balance = -1000; // illegal!`

With encapsulation: `setBalance(-1000)` can reject it.

Principle: hide implementation, expose behavior.

Code

```
// Without encapsulation:  
account.balance = -1000; // illegal!
```

Teaching focus

- Protects internal state from invalid modifications.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Private Fields

Concept

Declare fields private to prevent direct access.

External code cannot read/write balance directly.

Code

```
public class BankAccount {  
    private String owner;  
    private double balance;  
}
```

Teaching focus

- Declare fields private to prevent direct access.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Getter Methods

Concept

Public methods to read private fields.

Code

```
private fields.  
public double getBalance() {  
    return balance;  
}  
public String getOwner() {  
    return owner;  
}
```

Teaching focus

- Public methods to read private fields.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Setter Methods with Validation

Concept

Setters can enforce business rules.

Code

```
public void setBalance(double amount) {  
    if (amount < 0) {  
        throw new IllegalArgumentException("Negative  
balance");  
    }  
    this.balance = amount;  
}
```

Teaching focus

- Setters can enforce business rules.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Immutable Fields with final

Concept

Use final to make a field immutable after construction.

No setter: accountNumber cannot be changed.

Code

```
private final String accountNumber;  
public BankAccount(String accNum) {  
    this.accountNumber = accNum;  
}
```

Teaching focus

- Use final to make a field immutable after construction.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Benefits of Encapsulation

Concept

Validation logic in one place — easy to maintain.

Implementation can change without breaking callers.

Prevents inconsistent state (e.g., negative balance).

Core pillar of OOP: encapsulation, inheritance, polymorphism.

Teaching focus

- Validation logic in one place, easy to maintain.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

JavaBeans Naming Convention

Concept

Standard naming for getters/setters.

Field: `private String name;`

Getter: `public String getName() { return name; }`

Setter: `public void setName(String name) { this.name = name; }`

Followed by frameworks (Spring, Hibernate, etc.)

Code

```
private String name;  
public String getName() { return name; }  
public void setName(String name) {  
    this.name = name;  
}
```

Teaching focus

- Standard naming for getters/setters.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Read-Only Properties

Concept

Provide getter but no setter.

```
private final long id;
```

```
public long getId() { return id; }
```

```
// No setId() — id cannot be changed after creation.
```

Common pattern for unique identifiers.

Code cue

```
1 private final long id;  
2 public long getId() { return  
  id; }
```

Teaching focus

- Provide getter but no setter.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

Write-Only Properties

Concept

Rare: setter but no getter (e.g., passwords).

```
private String passwordHash;  
public void setPassword(String pw) {  
    this.passwordHash = hash(pw);  
}  
// No getPasswordHash() — security!
```

Code

```
private String passwordHash;  
public void setPassword(String pw) {  
    this.passwordHash = hash(pw);  
}
```

Teaching focus

- Rare: setter but no getter (e.g., passwords).
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

A Meaningful toString() Implementation

Concept

Useful toString using private fields via getter logic.

@Override

```
public String toString() {  
    return "Student{name='" + name + "', age=" + age + "'}";  
}  
System.out.println(s); // Student{name='Alice', age=20}
```

Defensive Copying

Concept

Return a copy of mutable fields in getters.

Prevents callers from mutating internal state.

Code cue

```
public int[] getScores() {  
    return scores.clone(); // not  
    the original array  
}
```

Records in Modern Java

Concept

Compact syntax for immutable data classes.

```
record Point(int x, int y) {}
```

Auto-generates: constructor, getters, equals, toString.

Code

```
record Point(int x, int y) {}  
Point p = new Point(3, 4);
```

Teaching focus

- Compact syntax for immutable data classes.
- Encapsulation protects invariants and makes object-oriented code safer to evolve.

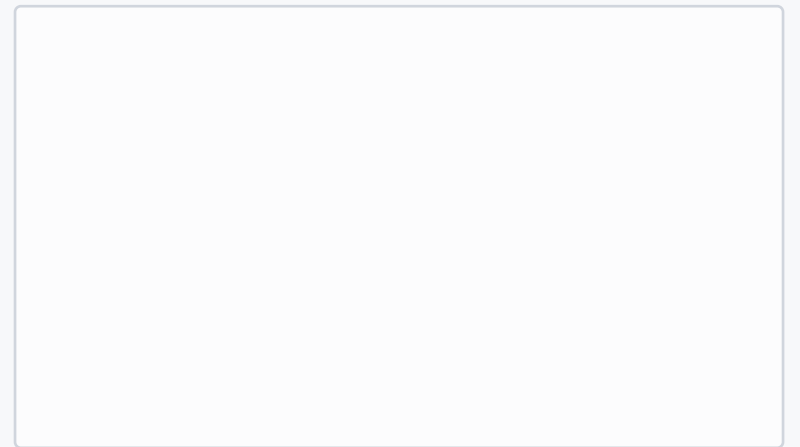
Encapsulation Checkpoint

Concept

Q: What access modifier hides a field from outside?

Q: How do you expose a private field safely?

Q: Can a setter reject a value?



Design Exercise

Concept

[iris-big-data/java/tp4.md at master · elomedah/iris-big-data](#)



Teaching focus

- Before coding, identify the class responsibilities, fields, and method contracts.

Module 7 · Inheritance

Inheritance, overriding, abstract classes, casting, and when to choose composition instead.

Selected themes

- Inheritance as an is-a Relationship
- Using extends
- Using super
- Inherited Members

Inheritance as an is-a Relationship

Concept

A class can extend another to reuse its fields and methods.

Subclass (child) inherits from superclass (parent).

Keyword: extends

```
class Student extends Person { ... }
```

'Is-a' relationship: a Student IS-A Person.

Code

```
1 class can extend another to  
  reuse its fields and methods.  
2 class Student extends Person {  
  ... }
```

Teaching focus

- A class can extend another to reuse its fields and methods.
- Inheritance captures specialization, but it must be used only when a true is-a relationship exists.

Using extends and Using super

Concept

```
public class Student extends Person {
```

Access the parent class constructor or methods.

```
public Student(String name, int id) {  
  super(name); // calls Person(String)  
  this.studentId = id;  
}
```

super() must be the first statement in child constructor.

Code

```
public class Person {  
  protected String name;  
  public Person(String name) {  
    this.name = name;  
  }  
  public class Student extends Person {  
    private int studentId;  
  }  
}
```

Inherited Members

Concept

Student inherits all public/protected members of Person.

Student s = new Student("Alice", 1001);

s.name → Alice (inherited from Person)

s.studentId → 1001 (Student's own field)

s.getName() → inherited method from Person

Code cue

```
1 protected members of Person.  
2 new Student("Alice", 1001);
```

Teaching focus

- Student inherits all public/protected members of Person.
- Inheritance captures specialization, but it must be used only when a true is-a relationship exists.

Method Overriding

Concept

Child class provides its own version of a parent method.

Tells the compiler this method overrides a parent method.

If signature doesn't match parent, compile error occurs.

@Override

```
public String toString() { return  
name + " #" + id; }
```

Always use @Override prevents subtle bugs.

Java code

```
1 Child class provides its own version of a parent method.  
2 @Override annotation helps catch errors.  
3 class Animal { public String sound(){return "...";} }  
4 class Dog extends Animal {  
5     @Override  
6     public String sound(){ return "Woof"; }  
7 }
```

Calling the Parent Implementation with super

Concept

super.method() calls the parent's version.

Java allows multilevel inheritance.

Person → Student → GraduateStudent

GraduateStudent inherits from both.

Java does NOT support multiple inheritance (no extends A, B).

Use interfaces for multiple 'types'.

Code

```
class Professor extends Person {  
    @Override  
    public String toString() {  
        return "Prof. " + super.toString();  
    }  
}
```

protected Access and final Classes and Methods

Concept

protected members are accessible in subclasses.

final class: cannot be subclassed.

Code

```
public class Person {  
    protected String name; // accessible in Student  
}
```

```
final class: cannot be subclassed.  
final class ImmutablePoint { ... }  
// no extends allowed  
final method: cannot be overridden.  
public final void validate() { ... }  
String class is final in Java.
```

Abstract Classes

Concept

Cannot be instantiated; designed to be subclassed.

```
abstract class Shape {  
    abstract double area(); // no body  
    public void print() { System.out.println(area()); }  
}
```

Subclasses must implement all abstract methods.

Teaching focus

- Cannot be instantiated; designed to be subclassed.
- Inheritance captures specialization, but it must be used only when a true is-a relationship exists.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Abstract Methods

Concept

Declared without a body; forces subclasses to implement.

Formatted Java code

```
1 Declared without a body; forces subclasses to implement.
2 abstract double area();
3 class Circle extends Shape {
4     double r;
5     @Override
6     double area() { return Math.PI * r * r; }
7 }
```

The Root Object Class

Concept

Every Java class implicitly extends Object.

Object provides: toString(), equals(), hashCode(), getClass().

This is why you can always call toString() on any object.

Override these methods for meaningful behavior.

Code cue

```
1 class implicitly extends  
Object.
```

Testing Types with instanceof

Concept

Checks if an object is an instance of a class.

```
Animal a = new Dog();
```

```
if (a instanceof Dog) {
```

```
    Dog d = (Dog) a; // safe cast
```

```
    d.fetch();
```

```
}
```

Java 16+: `if (a instanceof Dog d) { d.fetch(); }`

Code cue

```
1 new Dog();  
2 if (a instanceof Dog) {  
3     Dog d = (Dog) a; // safe cast  
4     d.fetch();  
5     if (a instanceof Dog d) { d.fetch(); }
```

Upcasting

Concept

Treating a subclass object as its parent type.

Person p = new Student("Alice", 1001); // upcast

p.getName() → "Alice" (Person method OK)

p.studentId → ERROR (not visible via Person reference)

Upcasting is automatic and safe.

Code cue

```
1 class object as its parent  
type.  
2 new Student("Alice", 1001); //  
upcast
```

Downcasting

Concept

Treating a parent reference as the child type.

```
Person p = new Student("Bob", 2002);
```

```
Student s = (Student) p; // explicit cast
```

s.studentId → 2002 (OK!)

Fails with `ClassCastException` if object is not that type.

Code cue

```
1 new Student("Bob", 2002);  
2 Student s = (Student) p; //  
  explicit cast
```

An Example Inheritance Hierarchy

Concept

Person (name, age, toString())

↳ Student (studentId, getGrade())

↳ Professor (department, teach())

Both Student and Professor are Persons.

Allows treating them uniformly as Person objects.

Teaching focus

- Person (name, age, toString())
- Inheritance captures specialization, but it must be used only when a true is-a relationship exists.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Inheritance versus Composition

Concept

Inheritance = is-a: Student IS-A Person. ✓

Composition = has-a: Car HAS-A Engine. ✓

Don't use inheritance just to reuse code.

Ask: 'Is a X really a type of Y?'

Prefer composition over inheritance when unsure.

Inheritance: Summary

Concept

extends creates inheritance relationship.
super() calls parent constructor.
@Override for method overriding.
abstract classes force subclass implementation.

[iris-big-data/java/tp5.md at master · elomedah/iris-big-data](#)

Code cue

```
1 @Override for method  
   overriding.  
2 abstract classes force subclass  
   implementation.
```

Teaching focus

- extends creates inheritance relationship.
- Inheritance captures specialization, but it must be used only when a true is-a relationship exists.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 8 · Polymorphism and Interfaces

Polymorphism, interfaces, lambdas, functional interfaces, and extensible design with abstractions.

Selected themes

- for Loop: Counter-Controlled Iteration
- Polymorphism
- Runtime Polymorphism and Dynamic Dispatch
- Polymorphism with Arrays
- Overloading versus Overriding

Polymorphism

Concept

One interface, multiple implementations.

An object can be treated as its parent type.

The correct method is chosen at runtime.

```
Animal[] zoo = {new Dog(), new Cat(), new Bird()};
```

```
for (Animal a : zoo) a.makeSound(); // calls correct version
```

Code cue

```
1 new Dog(), new Cat(), new  
  Bird()};  
2 for (Animal a : zoo)  
  a.makeSound(); // calls correct  
  version
```

Teaching focus

- One interface, multiple implementations.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Runtime Polymorphism and Dynamic Dispatch

Concept

Method resolution happens at runtime, not compile time.

Also called dynamic dispatch or late binding.

Animal a = new Dog(); a.makeSound(); // calls Dog.makeSound

Even though reference type is Animal, Dog's method runs.

Code cue

```
1 new Dog(); a.makeSound(); //  
calls Dog.makeSound
```

Teaching focus

- Method resolution happens at runtime, not compile time.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Polymorphism with Arrays

Concept

Store different subclasses in a parent type array.

```
Shape[] shapes = {new Circle(5), new Rectangle(3,4)};
```

```
double total = 0;
```

```
for (Shape s : shapes) total += s.area();
```

Clean and extensible — add new shapes without changing loop.

Code cue

```
1 new Circle(5), new  
Rectangle(3,4)};  
2 double total = 0;  
3 for (Shape s : shapes) total +=  
s.area();  
4 new shapes without changing  
loop.
```

Teaching focus

- Store different subclasses in a parent type array.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Overloading versus Overriding

Concept

Overriding: same name, same parameters → runtime dispatch.
Overloading: same name, different parameters → compile time.
Polymorphism only applies to overriding, not overloading.
Overriding requires exact matching of signature.

Teaching focus

- Overriding: same name, same parameters → runtime dispatch.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Interfaces as Contracts

Concept

Defines a contract: what methods a class must implement.

```
interface Printable {  
    void print();  
}  
  
class Document implements Printable {  
    public void print() { System.out.println(content); }  
}
```

Code cue

```
1 class must implement.  
2 interface Printable {  
3     void print();  
4 class Document implements  
   Printable {  
5     public void print() {  
       System.out.println(content); }  
}
```

Teaching focus

- Defines a contract: what methods a class must implement.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Interface versus Abstract Class

Concept

Interface: pure contract, no state (before Java 8).

Abstract class: partial implementation, has state.

A class can implement MULTIPLE interfaces.

A class can extend only ONE class.

Use interface for behavior; abstract class for shared code.

Code cue

```
1 class can implement MULTIPLE  
  interfaces.  
2 class can extend only ONE  
  class.  
3 interface for behavior;  
  abstract class for shared code.
```

Teaching focus

- Interface: pure contract, no state (before Java 8).
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Implementing Multiple Interfaces

Concept

```
class Student implements Comparable<Student>, Printable {  
    public int compareTo(Student o) { ... }  
    public void print() { ... }  
}
```

Student must implement ALL methods from both interfaces.

Code cue

```
1 class Student implements  
    Comparable<Student>, Printable {  
2     public int compareTo(Student o)  
    { ... }  
3     public void print() { ... }
```

Teaching focus

- class Student implements Comparable<Student>, Printable {
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Natural Ordering with Comparable

Concept

Allows objects to be sorted naturally.

```
public int compareTo(Student other) {  
    return this.name.compareTo(other.name);  
}
```

`Arrays.sort(students);` // uses `compareTo` automatically

Code cue

```
1 public int compareTo(Student  
other) {  
2 return  
this.name.compareTo(other.name);  
3 Arrays.sort(students); // uses  
compareTo automatically
```

Teaching focus

- Allows objects to be sorted naturally.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Default Methods in Interfaces

Concept

Java 8+: interfaces can have default implementations.

```
interface Greetable {  
    default void greet() {  
        System.out.println("Hello!");  
    }  
}
```

Classes get this for free; can override if needed.

Code cue

```
1 interface Greetable {  
2     default void greet() {  
3         System.out.println("Hello!");  
4     }  
5 }
```

Classes get this for free; can override if needed.

Teaching focus

- Java 8+: interfaces can have default implementations.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Constants Declared in Interfaces

Concept

Fields in interfaces are implicitly public static final.

```
interface Config {  
    int MAX_SIZE = 100; // = public static final int  
    String VERSION = "1.0";  
}
```

Accessed as Config.MAX_SIZE from anywhere.

Code cue

```
1 public static final.  
2 interface Config {  
3     int MAX_SIZE = 100; // = public  
    static final int  
4     String VERSION = "1.0";
```

Teaching focus

- Fields in interfaces are implicitly public static final.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Functional Interfaces

Concept

Interface with exactly ONE abstract method.

Can be used with lambda expressions (Java 8+).

@FunctionalInterface

```
interface Transformer { int apply(int x); }
```

```
Transformer doubler = x -> x * 2;
```

```
doubler.apply(5) → 10
```

Code cue

```
1 interface Transformer { int  
  apply(int x); }  
2 Transformer doubler = x -> x *  
  2;
```

Teaching focus

- Interface with exactly ONE abstract method.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Lambda Expressions

Concept

Compact way to write a functional interface implementation.

Traditional: `Runnable r = new Runnable() { public void run(){...} };`

Lambda: `Runnable r = () -> System.out.println("Run!");`

Shorter, cleaner — used extensively in modern Java.

Code cue

```
1 interface implementation.  
2 new Runnable() { public void  
  run(){...} };  
3 System.out.println("Run!");
```

Teaching focus

- Compact way to write a functional interface implementation.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

if / else: Simple Decision Example

Concept

```
if(number > 10){
```

Teaching focus

- The condition must evaluate to a boolean value.
- Only one branch executes for a given input.
- The else block captures the alternative execution path.

Formatted Java code

```
1 if (number > 10){  
2     System.out.println("Large");  
3 }else{  
4     System.out.println("Small");  
5 }
```

Why Polymorphism Matters

Concept

Extensibility: add new types without changing existing code.

Flexibility: methods work on parent type → accept any subtype.

Example: `printAll(Shape[] s)` works for all `Shape` subclasses.

Core principle: Open/Closed Principle in SOLID design.

Code cue

```
1 new types without changing  
existing code.
```

Teaching focus

- Extensibility: add new types without changing existing code.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Casting in Polymorphic Code

Concept

Downcast when you need subtype-specific methods.

```
Shape s = new Circle(5);
```

```
if (s instanceof Circle c) { // Java 16+ pattern matching
```

```
    System.out.println(c.getRadius());
```

```
}
```

Code cue

```
1 new Circle(5);
2 if (s instanceof Circle c) { //
  Java 16+ pattern matching
3
  System.out.println(c.getRadius())
  ;
```

Teaching focus

- Downcast when you need subtype-specific methods.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Polymorphism with Abstract Classes

Concept

```
abstract class Vehicle {  
    abstract double fuelEfficiency();  
    public String summary() {  
        return "Efficiency: " + fuelEfficiency() + " L/100km";  
    }  
}
```

Car, Truck, Motorcycle each define fuelEfficiency().

Code cue

```
1 abstract class Vehicle {  
2     abstract double  
    fuelEfficiency();  
3     public String summary() {  
4         return "Efficiency: " +  
            fuelEfficiency() + " L/100km";  
    }
```

Teaching focus

- abstract class Vehicle {
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Polymorphism through Interfaces

Concept

```
interface Drawable { void draw();  
}
```

Teaching focus

- Read the example from top to bottom and identify the role of each line.
- Relate the syntax to the concept introduced by the slide title.
- Be ready to adapt this pattern during a practical laboratory session.

Formatted Java code

```
1 interface Drawable { void draw(); }  
2 class Circle implements Drawable { public void draw(){...} }  
3 class Square implements Drawable { public void draw(){...} }  
4 List<Drawable> items = new ArrayList<>();  
5 items.forEach(d -> d.draw()); // polymorphic call
```

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Polymorphism and the Strategy Pattern

Concept

Polymorphism enables the Strategy design pattern.

```
interface SortStrategy { void sort(int[] arr); }
```

```
class BubbleSort implements SortStrategy { ... }
```

```
class QuickSort implements SortStrategy { ... }
```

Swap strategy without changing the client code.

Code cue

```
1 interface SortStrategy { void  
  sort(int[] arr); }  
2 class BubbleSort implements  
  SortStrategy { ... }  
3 class QuickSort implements  
  SortStrategy { ... }
```

Teaching focus

- Polymorphism enables the Strategy design pattern.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Sealed Classes in Java 17+

Concept

Restricts which classes can extend a parent.
sealed class Shape permits Circle, Rectangle {}
Only Circle and Rectangle can extend Shape.
Allows exhaustive switch expressions.
Useful for domain modeling with known subtypes.

Code cue

```
1 class Shape permits Circle,  
Rectangle {
```

Teaching focus

- Restricts which classes can extend a parent.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Design Exercise: Zoo Simulator with Interfaces

Concept

Build animal hierarchy for a zoo simulator:

abstract Animal: name, makeSound(), eat()

Dog, Cat, Parrot extend Animal.

Interface Trainable: train() — Dog and Parrot implement it.

Store in Animal[]; polymorphically call eat() and makeSound().

Code cue

```
1 Store in Animal[];  
polymorphically call eat() and  
makeSound().
```

Teaching focus

- Build animal hierarchy for a zoo simulator:
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- Before coding, identify the class responsibilities, fields, and method contracts.

Polymorphism and Interfaces: Summary

Concept

Polymorphism: one interface, many behaviors.

Dynamic dispatch: runtime method resolution.

Interfaces: contracts for unrelated classes.

Lambdas: compact functional interface syntax.

Next: Exception handling.

Code cue

```
1 interface syntax.
```

Teaching focus

- Polymorphism: one interface, many behaviors.
- Polymorphism and interfaces make code extensible by programming to abstractions rather than concrete types.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 9 · Abstraction, Interfaces, and Enums

Advanced interface design, template method, comparators, iterables, enums, and the shape of reusable APIs.

Selected themes

- if / else: Simple Decision Example
- Designing Good Interfaces
- Capabilities through Interfaces versus Inheritance
- Designing Abstract Classes
- Template Method Pattern

if / else: Simple Decision Example

Concept

```
if(number > 10){
```

Teaching focus

- The condition must evaluate to a boolean value.
- Only one branch executes for a given input.
- The else block captures the alternative execution path.

Formatted Java code

```
1 if (number > 10){  
2     System.out.println("Large");  
3 }else{  
4     System.out.println("Small");  
5 }
```

Designing Good Interfaces

Concept

Interfaces define roles, not hierarchies.

Unrelated classes can share a role via an interface.

```
class Robot implements Walkable, Talkable { ... }
```

```
class Human implements Walkable, Talkable { ... }
```

No inheritance needed — just implement the contract.

Code cue

```
1 class Robot implements  
  Walkable, Talkable { ... }  
2 class Human implements  
  Walkable, Talkable { ... }
```

Teaching focus

- Interfaces define roles, not hierarchies.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Capabilities through Interfaces versus Inheritance

Concept

Inheritance: code reuse + type hierarchy.

Interface: type contract only (before Java 8).

Prefer interfaces when defining capabilities.

Example: Serializable, Runnable, Comparable.

These are behaviors, not family relationships.

Teaching focus

- Inheritance: code reuse + type hierarchy.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Designing Abstract Classes

Concept

Use abstract class when subclasses share code.

```
abstract class Animal {  
    protected String name; // shared state  
    abstract void makeSound(); // subclass defines  
    void breathe() { System.out.println("Breathing..."); }  
}
```

Code cue

```
1 abstract class when subclasses share  
   code.  
2 abstract class Animal {  
3     protected String name; // shared  
   state  
4     abstract void makeSound(); //  
   subclass defines  
5     System.out.println("Breathing...");  
}
```

Teaching focus

- Use abstract class when subclasses share code.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Template Method Pattern

Concept

Abstract class defines algorithm skeleton;
subclasses fill in specific steps.

```
abstract class DataProcessor {  
    final void process() { read(); process(); write(); }  
    abstract void read(); abstract void process();  
    void write() { /* default */ }  
}
```

Code cue

```
1 class defines algorithm  
   skeleton;  
2 abstract class DataProcessor {  
3     final void process() { read();  
      process(); write(); }  
4     abstract void read(); abstract  
      void process();  
5     void write() { /* default */ }
```

Teaching focus

- Abstract class defines algorithm skeleton;
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Defining a Simple Student Class

Concept

```
class Student{
```

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Formatted Java code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Default Methods Revisited

Concept

Provide default behavior without breaking existing code.

```
interface Collection {  
    default boolean isEmpty() { return size() == 0; }  
    int size(); // still abstract  
}
```

All Collection implementations get isEmpty() for free.

Code cue

```
1 interface Collection {  
2     return size() == 0; }  
3 int size(); // still abstract
```

Teaching focus

- Provide default behavior without breaking existing code.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Static Methods in Interfaces

Concept

Utility methods on the interface itself (Java 8+).

```
interface StringUtils {  
    static boolean isBlank(String s) {  
        return s == null || s.trim().isEmpty();  
    }  
}  
StringUtils.isBlank("") → true
```

Code cue

```
1 interface itself (Java 8+).  
2 interface StringUtils {  
3     static boolean isBlank(String  
4         s) {  
5         return s == null ||  
6         s.trim().isEmpty();  
7     }  
8 }
```

Teaching focus

- Utility methods on the interface itself (Java 8+).
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Marker Interfaces

Concept

Interface with no methods — marks a class for special handling.

```
interface Serializable {} // from java.io
```

```
class MyData implements Serializable { ... }
```

JVM recognizes Serializable and enables serialization.

Modern approach: use annotations instead.

Code cue

```
1 class for special handling.  
2 interface Serializable {} //  
  from java.io  
3 class MyData implements  
  Serializable { ... }
```

Teaching focus

- Interface with no methods — marks a class for special handling.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Choosing Between Abstract Class and Interface

Concept

Use abstract class when:

- Classes share state (fields) or code.
- Strong is-a relationship exists.

Use interface when:

- Defining capability/role.
- Multiple unrelated classes need the same contract.

Code cue

```
1 abstract class when:  
2 interface when:
```

Teaching focus

- Use abstract class when:
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Exercise: Designing a Shape Interface

Concept

```
interface Shape {  
    double area();  
    double perimeter();  
    default void describe() {  
        System.out.printf("Area=%.2f Perim=%.2f%n", area(), perimeter());  
    }  
}
```

Code cue

```
1 interface Shape {  
2     double area();  
3     double perimeter();  
4     default void describe() {  
5         System.out.printf("Area=%.2f  
Perim=%.2f%n", area(),  
perimeter());  
}
```

Teaching focus

- interface Shape {
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- Before coding, identify the class responsibilities, fields, and method contracts.

Circle as a Shape Implementation

Concept

```
class Circle implements Shape {  
    private double radius;  
    public Circle(double r) { this.radius = r; }  
    public double area() { return Math.PI * radius * radius; }  
    public double perimeter() { return 2 * Math.PI * radius; }  
}
```

Code cue

```
1 class Circle implements Shape {  
2     private double radius;  
3     public Circle(double r) {  
4         this.radius = r; }  
5     public double area() { return  
6         Math.PI * radius * radius; }  
7     public double perimeter() {  
8         return 2 * Math.PI * radius; }  
9 }
```

Teaching focus

- class Circle implements Shape {
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Rectangle as a Shape Implementation

Concept

```
class Rectangle implements Shape {  
    private double w, h;  
    public Rectangle(double w, double h) { this.w=w; this.h=h; }  
    public double area() { return w * h; }  
    public double perimeter() { return 2*(w+h); }  
}
```

Code cue

```
1 class Rectangle implements Shape {  
2     private double w, h;  
3     public Rectangle(double w, double  
4         h) { this.w=w; this.h=h; }  
5     public double area() { return w *  
6         h; }  
7     public double perimeter() { return  
8         2*(w+h); }  
9 }
```

Teaching focus

- class Rectangle implements Shape {
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Polymorphic Use of Shape Objects

Concept

```
List<Shape> shapes = new ArrayList<>();  
shapes.add(new Circle(5));  
shapes.add(new Rectangle(3, 4));  
for (Shape s : shapes) s.describe();  
// Area=78.54 Perim=31.42  
// Area=12.00 Perim=14.00
```

Code cue

```
1 List<Shape> shapes = new  
ArrayList<>();  
2 new Circle(5));  
3 new Rectangle(3, 4));  
4 for (Shape s : shapes)  
s.describe();
```

Teaching focus

- `List<Shape> shapes = new ArrayList<>();`
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

if / else: Simple Decision Example

Concept

```
if(number > 10){
```

Teaching focus

- The condition must evaluate to a boolean value.
- Only one branch executes for a given input.
- The else block captures the alternative execution path.

Formatted Java code

```
1 if (number > 10){  
2     System.out.println("Large");  
3 }else{  
4     System.out.println("Small");  
5 }
```

Comparable versus Comparator

Concept

Comparable: natural order defined in the class itself.

implements Comparable<T> → compareTo(T o)

Comparator: external ordering strategy.

```
Comparator<Student> byAge = (a,b) -> a.age - b.age;  
students.sort(byAge);
```

Code cue

```
1 class itself.  
2 Comparator<Student> byAge =  
  (a,b) -> a.age - b.age;  
3 students.sort(byAge);
```

Teaching focus

- Comparable: natural order defined in the class itself.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Making a Class Iterable

Concept

Implementing Iterable allows for-each usage.

```
class Range implements Iterable<Integer> {  
    public Iterator<Integer> iterator() { ... }  
}  
for (int n : new Range(1, 5)) System.out.println(n);
```

Code cue

```
1 class Range implements  
  Iterable<Integer> {  
2   public Iterator<Integer>  
    iterator() { ... }  
3   for (int n : new Range(1, 5))  
    System.out.println(n);
```

Teaching focus

- Implementing Iterable allows for-each usage.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Interface Segregation Principle

Concept

SOLID Principle: don't force classes to implement unused methods.

Bad: `interface Animal { fly(); swim(); run(); }`

Good: `interface Flyable { fly(); }`

`interface Swimmable { swim(); }`

Duck implements Flyable, Swimmable.

Code cue

```
1 interface Animal { fly();  
  swim(); run(); }  
2 interface Flyable { fly(); }  
3 interface Swimmable { swim(); }
```

Teaching focus

- SOLID Principle: don't force classes to implement unused methods.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Enumerated Types

Concept

Enums define a fixed set of constants.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }  
Day today = Day.WED;  
switch(today) {  
case MON, TUE, WED, THU, FRI -> "Weekday";  
case SAT, SUN -> "Weekend";  
}
```

Code cue

```
1 enum Day { MON, TUE, WED, THU,  
FRI, SAT, SUN }  
2 Day today = Day.WED;  
3 switch (today) {  
4 case MON, TUE, WED, THU, FRI ->  
"Weekday";  
5 case SAT, SUN -> "Weekend";
```

Teaching focus

- Enums define a fixed set of constants.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Enums with Fields and Methods

Concept

Enums can have fields and methods.

```
enum Planet {  
    EARTH(5.97e24, 6.371e6),  
    MARS(6.39e23, 3.389e6);  
    private final double mass, radius;  
    Planet(double m, double r) { mass=m; radius=r; }  
}
```

Code cue

```
1 enum Planet {  
2     MARS(6.39e23, 3.389e6);  
3     private final double mass,  
        radius;  
4     Planet(double m, double r) {  
        mass=m; radius=r; }  
}
```

Teaching focus

- Enums can have fields and methods.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Nested Interfaces

Concept

Interfaces can be declared inside a class.

```
class LinkedList {  
    interface Node<T> { T getValue(); Node<T> next(); }  
}
```

Used to scope helper interfaces near their usage.

Code cue

```
1 class LinkedList {  
2     interface Node<T> { T  
    getValue(); Node<T> next(); }  
}
```

Teaching focus

- Interfaces can be declared inside a class.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Design Exercise: Payment System Interface

Concept

Build a payment system:

```
interface Payment { void pay(double amount); String receipt(); }
```

```
class CreditCard implements Payment { ... }
```

```
class PayPal implements Payment { ... }
```

Process a list of Payment objects polymorphically.

Code cue

```
1 interface Payment { void  
  pay(double amount); String  
  receipt(); }  
2 class CreditCard implements  
  Payment { ... }  
3 class PayPal implements Payment  
  { ... }
```

Teaching focus

- Build a payment system:
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- Before coding, identify the class responsibilities, fields, and method contracts.

Abstraction, Interfaces, and Enums: Summary

Concept

Interfaces: contracts with default/static methods.
Abstract classes: shared state + template method.
Comparable vs Comparator sorting.
Enum: fixed constant sets with behavior.
Next: Exception handling.

Code cue

```
1 static methods.
```

Teaching focus

- Interfaces: contracts with default/static methods.
- Advanced abstractions, sorting strategies, and enums strengthen design quality and expressiveness.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 10 · Exception Handling

Checked and unchecked exceptions, custom exceptions, cleanup, diagnostics, and resilient error handling.

Selected themes

- for Loop: Counter-Controlled Iteration
- What Is an Exception?
- The Exception Hierarchy
- Checked versus Unchecked Exceptions
- Basic try / catch Handling

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

What Is an Exception?

Concept

An exception is an event that disrupts normal execution.
Caused by: dividing by zero, null access, bad input, file not found.
Without handling, the program crashes with a stack trace.
Java provides a structured way to handle exceptions.

Teaching focus

- An exception is an event that disrupts normal execution.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

The Exception Hierarchy

Concept

Throwable

- └─ Error (JVM errors — don't catch: OutOfMemoryError)
- └─ Exception
 - └─ RuntimeException (unchecked: NullPointerException, etc.)
 - └─ IOException, etc. (checked — must handle or declare)

Teaching focus

- Throwable
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Checked versus Unchecked Exceptions

Concept

Checked: compiler forces you to handle them.

IOException, SQLException, FileNotFoundException.

Unchecked (RuntimeException): optional handling.

NullPointerException, ArrayIndexOutOfBoundsException.

Unchecked = programming mistakes; checked = expected errors.

Code cue

```
1 Unchecked = programming  
mistakes; checked = expected  
errors.
```

Teaching focus

- Checked: compiler forces you to handle them.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Handling

Concept

Wrap risky code in try; catch the exception.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
}  
// prints: Error: / by zero
```

Code cue

```
1 Wrap risky code in try; catch  
  the exception.  
2 try {  
3     int result = 10 / 0;  
4 } catch (ArithmeticException e) {  
5     System.out.println("Error: " +  
      e.getMessage());
```

Teaching focus

- Wrap risky code in try; catch the exception.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Example

Concept

try{

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Formatted Java code

```
1 try{  
2     int x = 10/0;  
3 }catch (Exception e){  
4     System.out.println("Error");  
5 }
```

Multiple catch Blocks

Concept

Catch different exception types separately.

```
try {  
    int[] arr = new int[5];  
    arr[10] = 1; // ArrayIndexOutOfBoundsException  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Bad index");  
} catch (Exception e) {  
    System.out.println("General error");  
}
```

Code cue

```
1 try {  
2   int[] arr = new int[5];  
3   arr[10] = 1; //  
   ArrayIndexOutOfBoundsException  
4   catch  
   (ArrayIndexOutOfBoundsException e) {  
5     System.out.println("Bad index");  
6   catch (Exception e) {  
7     System.out.println("General error");
```

Teaching focus

- Catch different exception types separately.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

The finally Block

Concept

Always executes — used for cleanup (close file, connection).

```
try {  
    openFile();  
    process();  
} catch (IOException e) {  
    handleError(e);  
} finally {  
    closeFile(); // always runs  
}
```

Code cue

```
1 try {  
2   openFile();  
3   process();  
4   catch (IOException e) {  
5     handleError(e);  
6   closeFile(); // always runs
```

Teaching focus

- Always executes — used for cleanup (close file, connection).
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

try-with-resources

Concept

Java 7+: automatically closes AutoCloseable resources.

```
try (FileReader fr = new FileReader("file.txt")) {  
    // use fr  
} catch (IOException e) {  
    e.printStackTrace();  
}  
// fr.close() is called automatically
```

Code cue

```
1 new FileReader("file.txt")) {  
2 catch (IOException e) {  
3 e.printStackTrace();
```

Teaching focus

- Java 7+: automatically closes AutoCloseable resources.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Throwing Exceptions Explicitly

Concept

Manually throw an exception.

```
public void setAge(int age) {  
    if (age < 0 || age > 150) {  
        throw new IllegalArgumentException("Invalid age: " + age);  
    }  
    this.age = age;  
}
```

Code cue

```
1 throw an exception.  
2 public void setAge(int age) {  
3     if (age < 0 || age > 150) {  
4         throw new  
        IllegalArgumentException("Invalid  
        age: " + age);  
5     this.age = age;
```

Teaching focus

- Manually throw an exception.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Example

Concept

try{

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Formatted Java code

```
1 try{
2     int x = 10/0;
3 }catch (Exception e){
4     System.out.println("Error");
5 }
```

Declaring Exceptions with throws

Concept

Declares that a method may throw a checked exception.

```
public void readFile(String path) throws IOException {  
    FileReader fr = new FileReader(path);  
    // ...  
}
```

Callers must catch or also declare throws IOException.

Code cue

```
1 throw a checked exception.  
2 public void readFile(String  
  path) throws IOException {  
3   new FileReader(path);
```

Teaching focus

- Declares that a method may throw a checked exception.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Creating Custom Exceptions

Concept

Create domain-specific exceptions by extending Exception.

```
public class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(double amount) {  
        super("Insufficient funds: need " + amount);  
    }  
}
```

Code cue

```
1 public class  
   InsufficientFundsException  
   extends Exception {  
2 public  
   InsufficientFundsException(double  
   amount) {  
3 super("Insufficient funds: need  
   " + amount);
```

Teaching focus

- Create domain-specific exceptions by extending Exception.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Using a Custom Exception

Concept

```
public void withdraw(double amount) throws
InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException(amount - balance);
    }
    balance -= amount;
}
```

Code cue

```
1 public void withdraw(double
amount) throws
InsufficientFundsException {
2     if (amount > balance) {
3         throw new
InsufficientFundsException(amount
- balance);
4     balance -= amount;
```

Teaching focus

- public void withdraw(double amount) throws InsufficientFundsException {
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Exception Messages and Stack Traces

Concept

e.getMessage() → readable error description.
e.printStackTrace() → full method call history.
e.getCause() → the original cause (if wrapped).
Use these in catch blocks for debugging.

Teaching focus

- e.getMessage() → readable error description.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Example

Concept

try{

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Formatted Java code

```
1 try{
2     int x = 10/0;
3 }catch (Exception e){
4     System.out.println("Error");
5 }
```

Multi-Catch in Java 7+

Concept

Catch multiple exception types in one block.

```
try {  
    // risky operations  
} catch (IOException | SQLException e) {  
    System.out.println("Data error: " + e.getMessage());  
}
```

Code cue

```
1 try {  
2 catch (IOException |  
    SQLException e) {  
3 System.out.println("Data error:  
    " + e.getMessage());
```

Teaching focus

- Catch multiple exception types in one block.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Rethrowing Exceptions

Concept

Catch, log, then rethrow to let the caller handle it.

```
} catch (IOException e) {  
    logger.error("Failed to read", e);  
    throw e; // rethrow  
}
```

Or wrap: `throw new RuntimeException("Wrapped", e);`

Code cue

```
1 throw to let the caller handle  
   it.  
2 catch (IOException e) {  
3     logger.error("Failed to read",  
4                 e);  
4     throw e; // rethrow  
5     throw new  
   RuntimeException("Wrapped", e);
```

Teaching focus

- Catch, log, then rethrow to let the caller handle it.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Exception-Handling Best Practices

Concept

Never swallow exceptions silently: `catch (Exception e) {}`

Use specific exception types, not generic `Exception`.

Log or report exceptions before handling.

Use `finally` or `try-with-resources` for cleanup.

Custom exceptions improve API clarity.

Code cue

```
1 catch (Exception e) {
```

Teaching focus

- Never swallow exceptions silently: `catch (Exception e) {}`
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Assertions

Concept

`assert condition : message;`

Used during development to verify assumptions.

`assert balance >= 0 : "Balance must not be negative!";`

Disabled by default — enable with: `java -ea MyApp`

Don't use for input validation in production.

Code cue

```
1 assert condition : message;  
2 assert balance >= 0 : "Balance  
must not be negative!";
```

Teaching focus

- `assert condition : message;`
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Exceptions in Constructors

Concept

Constructors can throw exceptions to prevent bad state.

```
public Connection(String url) throws SQLException {  
    if (url == null) throw new IllegalArgumentException("Null URL");  
    this.conn = DriverManager.getConnection(url);  
}
```

Failed constructor leaves no partial object.

Code cue

```
1 throw exceptions to prevent bad  
state.  
2 public Connection(String url)  
throws SQLException {  
3 if (url == null) throw new  
IllegalArgumentException("Null  
URL");  
4 this.conn =  
DriverManager.getConnection(url);
```

Teaching focus

- Constructors can throw exceptions to prevent bad state.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Checked Exceptions inside Stream Pipelines

Concept

Lambda expressions cannot throw checked exceptions directly.
Workaround: wrap in unchecked exception.

```
list.forEach(item -> {  
  try { process(item); }  
  catch (IOException e) { throw new RuntimeException(e); }  
});
```

Code cue

```
1 throw checked exceptions  
   directly.  
2 list.forEach(item -> {  
3   try { process(item); }  
4   catch (IOException e) { throw  
   new RuntimeException(e); }  
5 });
```

Teaching focus

- Lambda expressions cannot throw checked exceptions directly.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Design Exercise: Exceptions in a BankAccount

Concept

Design a BankAccount with exceptions:
withdraw() throws InsufficientFundsException
deposit() throws IllegalArgumentException for negative amount
Demonstrate try-catch-finally with a transaction loop.
Print meaningful error messages.

Teaching focus

- Design a BankAccount with exceptions:
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- Before coding, identify the class responsibilities, fields, and method contracts.

Exception Handling: Summary

Concept

try-catch-finally for error handling.

Checked vs unchecked exceptions.

Custom exceptions for domain errors.

try-with-resources for AutoCloseable objects.

Next: Collections framework.

Teaching focus

- try-catch-finally for error handling.
- Robust exception handling prevents silent failures and makes APIs easier to maintain.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 11 · Collections and Streams

Collections, maps, sets, generics, iterators, comparators, and the first declarative stream pipelines.

Selected themes

- Defining a Simple Student Class
- Overview of the Collections Framework
- ArrayList
- Common ArrayList Operations
- LinkedList

Defining a Simple Student Class

Concept

```
class Student{
```

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Formatted Java code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Overview of the Collections Framework

Concept

java.util provides generic data structures.

List: ordered, duplicates allowed (ArrayList, LinkedList).

Set: unique elements, no duplicates (HashSet, TreeSet).

Map: key-value pairs (HashMap, TreeMap).

Queue/Deque: FIFO/LIFO structures.

Teaching focus

- java.util provides generic data structures.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

ArrayList

Concept

Resizable array — most commonly used List.

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Alice");
```

```
names.get(0) → "Alice"
```

```
names.size() → 1
```

```
names.remove("Alice");
```

Code cue

```
1 ArrayList<String> names = new  
ArrayList<>();  
2 names.add("Alice");  
3 names.remove("Alice");
```

Teaching focus

- Resizable array — most commonly used List.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Common ArrayList Operations

Concept

```
ArrayList<Integer> nums = new ArrayList<>();  
nums.add(10); nums.add(20); nums.add(30);  
nums.contains(20) → true  
nums.indexOf(30) → 2  
Collections.sort(nums); // sorts in place  
Collections.reverse(nums);
```

Code cue

```
1 ArrayList<Integer> nums = new  
  ArrayList<>();  
2 nums.add(10); nums.add(20);  
  nums.add(30);  
3 Collections.sort(nums); //  
  sorts in place  
4 Collections.reverse(nums);
```

Teaching focus

- `ArrayList<Integer> nums = new ArrayList<>();`
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

LinkedList

Concept

Doubly-linked list — efficient insert/remove at head/tail.

```
LinkedList<String> queue = new LinkedList<>();
```

```
queue.addFirst("A");
```

```
queue.addLast("B");
```

```
queue.removeFirst() → "A"
```

Use ArrayList for random access; LinkedList for queue/stack.

Code cue

```
1 List<String> queue = new  
LinkedList<>();  
2 queue.addFirst("A");  
3 queue.addLast("B");  
4 Use ArrayList for random  
access; LinkedList for  
queue/stack.
```

Teaching focus

- Doubly-linked list — efficient insert/remove at head/tail.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Defining a Simple Student Class

Concept

```
class Student{
```

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Formatted Java code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

HashSet

Concept

Unordered set — no duplicate elements.

```
HashSet<String> set = new HashSet<>();  
set.add("Alice"); set.add("Bob"); set.add("Alice");  
set.size() → 2 (duplicate rejected)  
set.contains("Bob") → true  
Iteration order is not guaranteed.
```

Code cue

```
1 HashSet<String> set = new  
  HashSet<>();  
2 set.add("Alice");  
  set.add("Bob"); set.add("Alice");
```

Teaching focus

- Unordered set — no duplicate elements.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

TreeSet

Concept

Sorted set — elements in natural order.

```
TreeSet<Integer> sorted = new TreeSet<>();  
sorted.add(5); sorted.add(1); sorted.add(3);  
System.out.println(sorted); // [1, 3, 5]  
sorted.first() → 1 | sorted.last() → 5
```

Backed by a Red-Black tree — $O(\log n)$ operations.

Code cue

```
1 TreeSet<Integer> sorted = new  
TreeSet<>();  
2 sorted.add(5); sorted.add(1);  
sorted.add(3);  
3 System.out.println(sorted); //  
[1, 3, 5]
```

Teaching focus

- Sorted set — elements in natural order.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

HashMap

Concept

Key-value store — $O(1)$ average access.

```
HashMap<String, Integer> scores = new HashMap<>();
```

```
scores.put("Alice", 95);
```

```
scores.put("Bob", 87);
```

```
scores.get("Alice") → 95
```

```
scores.containsKey("Bob") → true
```

Code cue

```
1 HashMap<String, Integer> scores  
= new HashMap<>();  
2 scores.put("Alice", 95);  
3 scores.put("Bob", 87);
```

Teaching focus

- Key-value store — $O(1)$ average access.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Iterating through a HashMap

Concept

```
// Iterate entries
for (Map.Entry<String, Integer> e : scores.entrySet()) {
    System.out.println(e.getKey() + " : " + e.getValue());
}
// Java 8+
scores.forEach((k, v) -> System.out.println(k + " = " + v));
```

Code cue

```
1 for (Map.Entry<String, Integer>
e : scores.entrySet()) {
2 System.out.println(e.getKey() +
" : " + e.getValue());
3 System.out.println(k + " = " +
v));
```

Teaching focus

- // Iterate entries
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Example

Concept

try{

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Formatted Java code

```
1 try{
2     int x = 10/0;
3 }catch (Exception e){
4     System.out.println("Error");
5 }
```

TreeMap

Concept

HashMap but keys are sorted in natural order.

```
TreeMap<String, Integer> map = new TreeMap<>();
```

```
map.put("Charlie", 70);
```

```
map.put("Alice", 95);
```

```
System.out.println(map); // {Alice=95, Charlie=70}
```

Useful when you need sorted key traversal.

Code cue

```
1 TreeMap<String, Integer> map =  
  new TreeMap<>();  
2 map.put("Charlie", 70);  
3 map.put("Alice", 95);  
4 System.out.println(map); //  
  {Alice=95, Charlie=70}
```

Teaching focus

- HashMap but keys are sorted in natural order.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Iterating over a List

Concept

```
for (String s : names) { System.out.println(s); }  
// With index:  
for (int i = 0; i < names.size(); i++) {  
    System.out.println(i + ": " + names.get(i));  
}  
// Java 8+:  
names.forEach(System.out::println);
```

Code cue

```
1 for (String s : names) {  
    System.out.println(s); }  
2 for (int i = 0; i <  
    names.size(); i++) {  
3     System.out.println(i + ": " +  
        names.get(i));  
4     System.out::println);
```

Teaching focus

- for (String s : names) {
 System.out.println(s); }
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

The Collections Utility Class

Concept

`java.util.Collections` provides static helpers.

Teaching focus

- Read the example from top to bottom and identify the role of each line.
- Relate the syntax to the concept introduced by the slide title.
- Be ready to adapt this pattern during a practical laboratory session.

Formatted Java code

```
1 java.util.Collections provides static helpers.  
2 Collections.sort(list);  
3 Collections.reverse(list);  
4 Collections.shuffle(list);  
5 Collections.max(list) / Collections.min(list)  
6 Collections.unmodifiableList(list); // read-only view
```

Generics Basics

Concept

`<T>` means 'any type' — enforced at compile time.

`ArrayList<String> names; // only Strings allowed`

`ArrayList<Integer> nums; // only Integers allowed`

Without generics: need to cast + risk `ClassCastException`.

Generics = type safety at compile time.

Code cue

```
1 ArrayList<String> names; //  
  only Strings allowed  
2 ArrayList<Integer> nums; //  
  only Integers allowed
```

Teaching focus

- `<T>` means 'any type' — enforced at compile time.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Basic try / catch Example

Concept

try{

Teaching focus

- The try block contains code that may fail at runtime.
- The catch block defines the recovery path when an exception occurs.
- Catching generic Exception is simple for demonstrations but too broad for production code.

Formatted Java code

```
1 try{
2     int x = 10/0;
3 }catch (Exception e){
4     System.out.println("Error");
5 }
```

Implementing a Stack with Deque

Concept

```
Deque<Integer> stack = new ArrayDeque<>();  
stack.push(1); stack.push(2); stack.push(3);  
stack.pop() → 3 (LIFO)  
stack.peek() → 2 (look without removing)  
Prefer Deque over legacy Stack class.
```

Code cue

```
1 Deque<Integer> stack = new  
  ArrayDeque<>();  
2 stack.push(1); stack.push(2);  
  stack.push(3);
```

Teaching focus

- `Deque<Integer> stack = new ArrayDeque<>();`
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Implementing a Queue with Deque

Concept

```
Queue<String> queue = new LinkedList<>();  
queue.offer("First");  
queue.offer("Second");  
queue.poll() → "First" (FIFO)  
queue.peek() → "Second" (look without removing)
```

Code cue

```
1 Queue<String> queue = new  
LinkedList<>();  
2 queue.offer("First");  
3 queue.offer("Second");
```

Teaching focus

- `Queue<String> queue = new LinkedList<>();`
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Immutable Collections with List.of and Map.of

Concept

Create immutable collections with factory methods.

```
List<String> list = List.of("A", "B", "C");
```

```
Set<Integer> set = Set.of(1, 2, 3);
```

```
Map<String,Integer> m = Map.of("a",1, "b",2);
```

These cannot be modified (no add/remove).

Code cue

```
1 List<String> list =  
  List.of("A", "B", "C");  
2 Set<Integer> set = Set.of(1, 2,  
  3);  
3 Map<String,Integer> m =  
  Map.of("a",1, "b",2);
```

Teaching focus

- Create immutable collections with factory methods.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Sorting with a Comparator

Concept

Sort a list with custom ordering.

```
List<Student> students = new ArrayList<>();  
students.sort(Comparator.comparing(Student::getName));  
students.sort(Comparator.comparingInt(Student::getAge).reversed()  
);
```

Method references make Comparator concise.

Code cue

```
1 List<Student> students = new  
  ArrayList<>();  
2  
  Comparator.comparing(Student::get  
    Name));  
3  
  Comparator.comparingInt(Student::  
    getAge).reversed());
```

Teaching focus

- Sort a list with custom ordering.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

for Loop: Counter-Controlled Iteration

Concept

```
for(int i=0;i<5;i++){
```

Teaching focus

- The initializer executes once before the loop begins.
- The condition is checked before each iteration.
- The update expression advances the counter after each pass.

Formatted Java code

```
1 for (int i=0;i<5;i++){  
2     System.out.println(i);  
3 }
```

Introduction to Streams

Concept

Java 8+ Streams process collections declaratively.

```
List<String> names = List.of("Alice","Bob","Charlie");  
names.stream()  
.filter(n -> n.startsWith("A"))  
.map(String::toUpperCase)  
.forEach(System.out::println); // ALICE
```

Code cue

```
1 List<String> names =  
  List.of("Alice","Bob","Charlie");  
2 .filter(n -> n.startsWith("A"))  
3 .map(String::toUpperCase)  
4 System.out::println); // ALICE
```

Teaching focus

- Java 8+ Streams process collections declaratively.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Terminal Operations in Streams

Concept

collect: gather results into a list/set/map.

count: number of elements.

findFirst: first element (Optional).

reduce: combine all elements.

```
List<String> filtered = names.stream().filter(n->n.length()>3).collect(Collectors.toList());
```

Code cue

```
1 List<String> filtered =  
  names.stream().filter(n->n.length()>3).collect(Collectors.toList());
```

Teaching focus

- collect: gather results into a list/set/map.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Design Exercise: Student Registry with Collections

Concept

Build a Student Registry:

Store students in `ArrayList<Student>`.

Use `HashMap<String,Student>` for lookup by ID.

Add, remove, find by name.

Sort by GPA with `Comparator`.

Print all with `forEach`.

Code cue

```
1 ArrayList<Student>.
2 HashMap<String,Student> for
  lookup by ID.
3 Comparator.
```

Teaching focus

- Build a Student Registry:
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- Before coding, identify the class responsibilities, fields, and method contracts.

Collections and Streams: Summary

Concept

ArrayList: ordered, dynamic array.

HashMap: key-value, fast access.

HashSet: unique elements.

Collections utility methods.

Next: Mini project — library or student management.

Teaching focus

- ArrayList: ordered, dynamic array.
- Choosing the right collection depends on ordering, uniqueness, lookup needs, and performance trade-offs.
- Review the vocabulary, syntax, and design rationale before moving on.

Module 12 · Project Synthesis and Next Steps

Project architecture, validation, testing, design recap, exam preparation, and next steps after Master 1.

Selected themes

- Defining a Simple Student Class
- Project Overview: Student Management System
- Project Architecture
- Student Domain Model
- StudentRegistry Service Class

Defining a Simple Student Class

Concept

```
class Student{
```

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Formatted Java code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Project Overview: Student Management System

Concept

Build a Student Management System.

Features: add, remove, search, display, sort students.

Use ArrayList and HashMap for storage.

Apply: OOP, encapsulation, exceptions, collections.

Output via console with formatted tables.

Teaching focus

- Build a Student Management System.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Project Architecture

Concept

Main.java — entry point, menu loop.

Student.java — data model (name, id, gpa, major).

StudentRegistry.java — manages the collection.

ConsoleUI.java — handles input/output.

Separation of concerns: model vs logic vs UI.

Teaching focus

- Main.java — entry point, menu loop.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Student Domain Model

Concept

```
public class Student {  
    private String name; private int id;  
    private double gpa; private String major;  
    public Student(String name, int id, double gpa, String major) {...}  
    // getters, setters, toString(), equals()  
}
```

Code cue

```
1 public class Student {  
2     private String name; private  
   int id;  
3     private double gpa; private  
   String major;  
4     public Student(String name, int  
   id, double gpa, String major)  
   {...}
```

Teaching focus

- public class Student {
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

StudentRegistry Service Class

Concept

```
public class StudentRegistry {  
    private List<Student> students = new ArrayList<>();  
    private Map<Integer,Student> byId = new HashMap<>();  
    public void add(Student s) { students.add(s); byId.put(s.getId(),s); }  
    public Student findById(int id) { return byId.get(id); }  
}
```

Code cue

```
1 public class StudentRegistry {  
2     private List<Student> students = new  
    ArrayList<>();  
3     private Map<Integer,Student> byId =  
    new HashMap<>();  
4     public void add(Student s) {  
        students.add(s);  
        byId.put(s.getId(),s); }  
5     public Student findById(int id) {  
        return byId.get(id); }  
}
```

Teaching focus

- public class StudentRegistry {
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

ArrayList: Creating and Populating a List

Concept

```
ArrayList<String> list = new  
ArrayList<>();
```

Teaching focus

- ArrayList is a resizable collection from java.util.
- The generic type parameter String enforces compile-time type safety.
- add(...) appends elements at the end of the list.

Formatted Java code

```
1 ArrayList<String> list = new ArrayList<>();  
2 list.add("Alice");  
3 list.add("Bob");
```

Search Functionality with Streams

Concept

```
public List<Student> searchByName(String query) {  
    return students.stream()  
        .filter(s -> s.getName().toLowerCase()  
            .contains(query.toLowerCase()))  
        .collect(Collectors.toList());  
}
```

Code cue

```
1 public List<Student>  
   searchByName(String query) {  
2     return students.stream()  
3     .filter(s ->  
       s.getName().toLowerCase()  
4     .collect(Collectors.toList()));
```

Teaching focus

- `public List<Student> searchByName(String query) {`
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Sorting Students by GPA

Concept

```
public List<Student> sortByGpa() {  
    return students.stream()  
        .sorted(Comparator.comparingDouble(Student::getGpa).reversed())  
        .collect(Collectors.toList());  
}  
// Returns new list sorted by GPA desc
```

Code cue

```
1 public List<Student>  
sortByGpa() {  
2 return students.stream()  
3  
Comparator.comparingDouble(Student  
t::getGpa).reversed())  
4 .collect(Collectors.toList());  
5 new list sorted by GPA desc
```

Teaching focus

- `public List<Student> sortByGpa() {`
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Project-Level Input Validation and Exceptions

Concept

Validate inputs to prevent bad state.

```
if (findById(id) != null) throw new DuplicateIdException(id);
```

```
if (gpa < 0 || gpa > 4.0) throw new IllegalArgumentException("Invalid GPA");
```

Wrap Scanner input in try-catch for NumberFormatException.

Show user-friendly error messages.

Code cue

```
1 if (findById(id) != null) throw  
  new DuplicateIdException(id);  
2 if (gpa < 0 || gpa > 4.0) throw  
  new  
  IllegalArgumentException("Invalid  
  GPA");  
3 Scanner input in try-catch for  
  NumberFormatException.
```

Teaching focus

- Validate inputs to prevent bad state.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Designing a Console Menu

Concept

while (true) {

Teaching focus

- Read the example from top to bottom and identify the role of each line.
- Relate the syntax to the concept introduced by the slide title.
- Be ready to adapt this pattern during a practical laboratory session.

Formatted Java code

```
1 while (true) {  
2     System.out.println("1. Add Student");  
3     System.out.println("2. Find by ID");  
4     System.out.println("3. List All");  
5     System.out.println("0. Exit");  
6     int choice = sc.nextInt();  
7     // switch (choice) { ... }  
8 }
```

ArrayList: Creating and Populating a List

Concept

```
ArrayList<String> list = new  
ArrayList<>();
```

Teaching focus

- ArrayList is a resizable collection from java.util.
- The generic type parameter String enforces compile-time type safety.
- add(...) appends elements at the end of the list.

Formatted Java code

```
1 ArrayList<String> list = new ArrayList<>();  
2 list.add("Alice");  
3 list.add("Bob");
```

Formatted Tabular Output

Concept

```
System.out.printf("%-5s %-20s %-10s %4s%n",
"ID","Name","Major","GPA");
System.out.println("-".repeat(42));
for (Student s : students) {
System.out.printf("%-5d %-20s %-10s %4.2f%n",
s.getId(), s.getName(), s.getMajor(), s.getGpa());
}
```

Code cue

```
1 System.out.printf("%-5s %-20s %-10s
%4s%n", "ID", "Name", "Major", "GPA");
2 System.out.println("-".repeat(42));
3 for (Student s : students) {
4 System.out.printf("%-5d %-20s %-10s
%4.2f%n",
5 s.getId(), s.getName(),
s.getMajor(), s.getGpa());
```

Teaching focus

- `System.out.printf("%-5s %-20s %-10s %4s%n", "ID","Name","Major","GPA");`
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Testing the Application

Concept

Add 5+ sample students with varied GPAs and majors.

Test: findById (existing and non-existing IDs).

Test: searchByName with partial matches.

Test: sortByGpa descending.

Test: error handling with invalid GPA or duplicate ID.

Teaching focus

- Add 5+ sample students with varied GPAs and majors.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Design Patterns Recap

Concept

Singleton: only one instance (e.g., StudentRegistry).

Factory: create objects without specifying class.

Strategy: swappable sorting or search algorithms.

Observer: notify UI when data changes.

You have used these patterns implicitly throughout.

Teaching focus

- Singleton: only one instance (e.g., StudentRegistry).
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Modern Java Features Recap

Concept

Lambda: (a, b) -> a.compareTo(b)

Method reference: Student::getName

Stream API: filter, map, collect, reduce.

Optional<T>: avoid null with Optional.ofNullable(x).

var (Java 10+): var list = new ArrayList<String>();

Code cue

```
1 Lambda: (a, b) ->  
  a.compareTo(b)  
2 Method reference:  
  Student::getName  
3 new ArrayList<String>();
```

Teaching focus

- Lambda: (a, b) -> a.compareTo(b)
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

if / else: Simple Decision Example

Concept

```
if(number > 10){
```

Teaching focus

- The condition must evaluate to a boolean value.
- Only one branch executes for a given input.
- The else block captures the alternative execution path.

Formatted Java code

```
1 if (number > 10){  
2     System.out.println("Large");  
3 }else{  
4     System.out.println("Small");  
5 }
```

OOP Principles Recap

Concept

Encapsulation: private fields + getters/setters.

Inheritance: extends reuses and specializes code.

Polymorphism: one reference, many behaviors.

Abstraction: hide complexity behind interfaces.

SOLID: Single Responsibility, Open/Closed, etc.

Code cue

```
1 private fields +  
getters/setters.
```

Teaching focus

- Encapsulation: private fields + getters/setters.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Exam Preparation Advice

Concept

Explain the difference between interface and abstract class.
Write a class with proper encapsulation from scratch.
Show inheritance: Person → Student → GraduateStudent.
Demonstrate try-catch-finally with custom exception.
Sort an ArrayList with Comparator.

Code cue

```
1 interface and abstract class.  
2 class with proper encapsulation  
  from scratch.  
3 Comparator.
```

Teaching focus

- Explain the difference between interface and abstract class.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Common Exam Mistakes

Concept

Comparing Strings with `==` instead of `.equals()`.

Forgetting `super()` in subclass constructor.

Not handling exceptions that are checked.

Confusing overloading (compile-time) vs overriding (runtime).

Using `static` for instance behavior.

Code cue

```
1 class constructor.  
2 static for instance behavior.
```

Teaching focus

- Comparing Strings with `==` instead of `.equals()`.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Final Project Checklist

Concept

- ✓ At least 3 classes (model, registry, main)
- ✓ Encapsulation: all fields private with getters/setters
- ✓ Inheritance or interface used
- ✓ Exception handling with custom exception
- ✓ Collections: ArrayList + HashMap
- ✓ User interaction via console

Code cue

```
1 private with getters/setters  
2 interface used
```

Teaching focus

- ✓ At least 3 classes (model, registry, main)
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Defining a Simple Student Class

Concept

```
class Student{
```

Teaching focus

- The class declares fields that represent the state of each Student object.
- Without methods, the class is a simple data model.
- This pattern introduces the distinction between a class blueprint and an object instance.

Formatted Java code

```
1 class Student{  
2     String name;  
3     int age;  
4 }
```

Alternative Project: Library Management System

Concept

Alternative project: Library Management System.

Classes: Book (isbn, title, author), Library (collection), Member.

Features: addBook, borrowBook, returnBook, searchByTitle.

Exception: BookUnavailableException, DuplicateISBNException.

Same structure, different domain.

Teaching focus

- Alternative project: Library Management System.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Code Review Checklist

Concept

Naming: meaningful names, camelCase for methods/vars.

Comments: Javadoc on public methods.

Error handling: no empty catch blocks.

Collections: use generics (not raw types).

DRY: no repeated code — extract to methods.

Code cue

```
1 public methods.
```

Teaching focus

- Naming: meaningful names, camelCase for methods/vars.
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Next Steps Beyond Master 1

Concept

Java frameworks: Spring Boot (REST APIs, microservices).
Build tools: Maven / Gradle.
Testing: JUnit 5, Mockito.
Databases: JPA / Hibernate (ORM).
Concurrency: threads, `CompletableFuture`, `ExecutorService`.

Teaching focus

- Java frameworks: Spring Boot (REST APIs, microservices).
- The final module synthesizes language features into a coherent, maintainable Java application.
- For assessment, be ready to explain the syntax, purpose, and a representative use case.

Course Summary and Continuation

Concept

12 weeks covering: environment → OOP → collections → project.

Core skills: classes, inheritance, polymorphism, exceptions.

Practical focus: every concept backed by working code.

Foundation for: Spring Boot, Android, enterprise Java.

Congratulations — you now write real Java!

Teaching focus

- 12 weeks covering: environment → OOP → collections → project.
- The final module synthesizes language features into a coherent, maintainable Java application.
- Review the vocabulary, syntax, and design rationale before moving on.